Lecture 12: Floating Point Numbers

Leili Rafiee Sevyeri

Based on lecture notes by me and many previous CS335 instructors

Fall 2026
Cheriton School of Computer Science
University of Waterloo

Announcements:

1. As mentioned before, there will be no class on the day of the midterm, Wednesday October 28. Instead I'll run an office hour during that timer. Please drop by to my office, MC 6431, if you have any questions.
2. Use the scheduled office hours for both A2 and the midterm. Details are announced on Piazza.
3. If you declared a short-term absence for A2, please note that the weight of A2 will be transferred to other assignments.
4. I have uploaded the solutions of In-Class Activities on D2L.
5. I'll upload the solution of A2 after 5pm today.
6. There is post on Piazza about the cheat sheet (post @42). Please feel free to contribute there :)

Measuring Error

If $x_{exact} = 10$, then the ratio is $\frac{10}{10} = 1 = 100\%$ error.

Our algorithms will compute approximate solutions to problems.

Difference between ...

x_{exact} = true solution, and

x_{approx} = approximate solution,

... gives the error of an algorithm.

If the absolute value of this difference is 10, is this a big error?

We distinguish between absolute error and relative error.

If $x_{exact} = 10^6$, then the ratio of the error to the value is $\frac{10}{10^6} = 0.00001 \rightarrow$ small.

End of Lecture 11

Absolute vs. Relative Error

Absolute error:

$$E_{abs} = |x_{exact} - x_{approx}|$$

Relative error:

$$E_{rel} = \frac{|x_{exact} - x_{approx}|}{|x_{exact}|}$$

Example

Let $x_{exact} = 220$, $x_{approx} = 198$

Compute E_{abs} and E_{rel}.

$E_{abs} = |220 - 198| = 22$

$E_{rel} = \frac{22}{|220|} = 0.1$

What does each tell us?

Note: sometimes we'll also consider the *signed* error, i.e. without abs. values here.

Floating point, F v.s. Reals, $\mathbb{R}$.

How much can a real number and its FP counterpart differ? i.e. how far does rounding/truncation move the input real number?

For FP system F, rel. err. between $x \in \mathbb{R}$, and its FP approximation, $fl(x)$, has a bound, ϵ_{mach}.

$$E_{rel} = \frac{|x - fl(x)|}{|x|} \leq \epsilon_{mach}, \quad \text{for a positive value of } \epsilon_{mach}.$$

$$|x - fl(x)| \leq |x| \, \epsilon_{mach}$$

$$|x| - |fl(x)| \leq |x - fl(x)|$$

$$\Rightarrow |x| - |fl(x)| \leq \epsilon_{mach} \cdot |x|$$

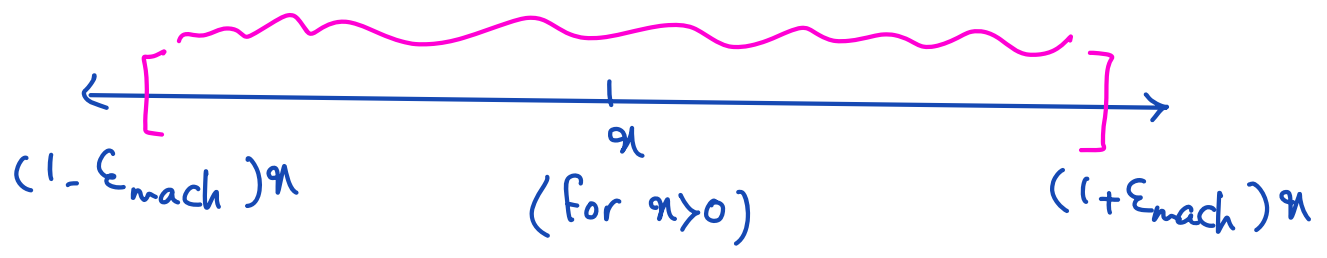
$$\Rightarrow (1 - \varepsilon_{mach})|x| \leq |fl(x)| \qquad (1)$$

$$|fl(x)| \leq (1 + \varepsilon_{mach})|x| \qquad (2)$$

from (1) and (2) :

$$(1 - \varepsilon_{mach})|x| \leq |fl(x)| \leq (1 + \varepsilon_{mach})|x|$$

i.e., $fl(x)$ is in a neighbourhood of x, somewhere!



Machine Epsilon

This maximum relative error, ϵ_{mach}, for a FP system is called *machine epsilon* or *unit round-off error.*

It is defined as the smallest value such that $fl(1 + \epsilon_{\text{mach}}) > 1$ under the given floating point system.

Let's look at a specific system to better understand this ...

$F = \{\beta, t, L, U\}$, with chopping. What is machine epsilon?

$1 = 0.100\ldots00 \times \beta^1$ (1)

$1 + \epsilon_{mach} = 0.100\ldots01 \times \beta^1$ (2)

$(2) - (1) = \epsilon_{mach} = 0.00\ldots01 \times \beta^1 = \beta^{-t} \times \beta^1 = \beta^{1-t}$

$x = 0.0001$, $t = 4$

$\beta = 10$

$0.0001 = 10^{-4} = \beta^{-4} = \beta^{-t}$

Machine Epsilon

We have the rule $fl(x) = x(1+\delta)$ for some $|\delta| \leq \epsilon_{\text{mach}}$.

$\delta = \frac{fl(x) - x}{x}$, $|\delta| \leq \epsilon_{\text{mach}}$

$fl(x) - x = \delta x \Longrightarrow fl(x) = x(1+\delta)$

For an FP system (β, t, L, U) with ...

- rounding to nearest: $\epsilon_{\text{mach}} = \frac{1}{2}\beta^{1-t}$.
- truncation: $\epsilon_{\text{mach}} = \beta^{1-t}$.

Example

What is ϵ_{mach} for $F = \{\beta = 10, t = 3, L = -5, U = 5\}$?

$\epsilon_{\text{mach}} = \frac{1}{2}\beta^{1-t} = \frac{1}{2}10^{-2} = 5 \times 10^{-3}$ with rounding to nearest. For truncation: $\epsilon_{\text{mach}} = \beta^{1-t} = 10^{-2}$

Arithmetic with Floating Point

$w \oplus z = fl(w+z)$

$fl(x) = x(1+\delta), \quad |\delta| \leq \epsilon_{mach}$

What guarantees are there on a FP arithmetic operation?

We requires that for $w, z \in F$,

$$w \oplus z = fl(w+z) = (w+z)(1+\delta)$$

where $\oplus$ denotes floating point addition.

again, with $|\delta| \leq \epsilon_{\text{mach}}$ (assuming no over/underflows occurs.)

i.e., it is the same as the floating point approximation of the exact real result.

$fl(w+z) = (w+z)(1+\delta) = w \oplus z, \quad |\delta| \leq \epsilon_{mach}$

Floating Point Arithmetic

$a+b+c = (a+b)+c = a+(b+c) \longrightarrow \mathbb{R}$ ✓

Warning: This rule only applies to individual FP operations! Not sequences of operations.

So: It is **not** generally true that

$$(a \oplus b) \oplus c \neq a \oplus (b \oplus c) \neq fl(a+b+c)$$

Result is order-dependent; associativity is broken!

Nonetheless, FP can still be (very!) useful if we understand its behaviour and limitations.

FP arithmetic examples

e.g. $F = \{\beta = 10, t = 4, L = -10, U = 10\}$, rounding arithmetic, with $x = 1235$, $y = -1231$ and $z = 56780$.

$x \oplus y =$ $fl(1235 - 1231) = fl(4) = 0.4000 \times 10^{1} = 4$

$(x \oplus y) \oplus z =$ $fl(4 + 56780) = fl(56784) = 0.56784 \times 10^{5}$ ($t=4$)

$= 0.5678 \times 10^{5} = 56780$

$y \oplus z =$ $fl(-1231 + 56780) = fl(55549) = 0.55549 \times 10^{5}$

$= 0.5555 \times 10^{5} = 55550$

$x \oplus (y \oplus z) =$ $fl(1235 + 55550) = fl(56785)$

$= 0.56785 \times 10^{5} = 0.5679 \times 10^{5} = 56790$

Round-Off Error Analysis, Stability

Floating Point Round-Off Error Analysis

We can use our rules for machine epsilon, relative error of an FP arithmetic operation, and the $fl(\cdot)$ operation to see how error accumulates...

Let's consider adding two **real** numbers $x, y \in \mathbb{R}$.

Since they are reals, we first have to convert them to FP representation, *then* perform FP addition:

$$x \oplus y = fl(x) \oplus fl(y) = fl(fl(x) + fl(y))$$

Floating Point Round-Off Error Analysis

Error of $x \oplus y$

Recall $fl(x) = x(1+\delta)$ for some δ, where $|\delta| \leq \varepsilon_{mach}$.

$$x \oplus y = fl(fl(x) + fl(y))$$

$$= fl(x(1+\delta_1) + y(1+\delta_2))$$

$$= (1+\delta_3)(x(1+\delta_1) + y(1+\delta_2))$$

$$\text{Abs. Err} = |x \oplus y - (x+y)|$$

$$= |(x + x\delta_1 + y + y\delta_2)(1+\delta_3) - x - y|$$

Floating Point Round-Off Error Analysis

$$= \left| \delta_1 x + \delta_2 y + \delta_3 x + \delta_1 \delta_3 x + \delta_3 y + \delta_2 \delta_3 y \right|$$

Apply tri. inequality : $|a+b| \leq |a| + |b|$

and $|a||b| = |ab|$

$$\text{Abs. Err} \leq \left| x(\delta_1 + \delta_3 + \delta_1 \delta_3) \right| + \left| y(\delta_2 + \delta_3 + \delta_2 \delta_3) \right|$$

$$\leq |x| \left(|\delta_1| + |\delta_3| + |\delta_1||\delta_3| \right)$$

$$+ |y| \left(|\delta_2| + |\delta_2| + |\delta_2||\delta_3| \right)$$

Apply $|\delta_i| \leq \varepsilon_{mach}$.

$$\text{Abs. Err} \leq |x|\left(2\varepsilon_{mach} + \varepsilon_{mach}^2\right) + |y|\left(2\varepsilon_{mach} + \varepsilon_{mach}^2\right)$$

$$\leq (|x| + |y|)\left(2\varepsilon_{mach} + \varepsilon_{mach}^2\right)$$

Since ε_{mach} is small, we assume $\varepsilon_{mach}^2 \ll \varepsilon_{mach}$, so often ignore ε_{mach}^2.

$$\text{Abs. Err} \leq (|x| + |y|)\, 2\varepsilon_{mach}$$

$$\text{Rel. Err} \leq \frac{\text{Abs. Err}}{|x+y|} \leq 2\varepsilon_{mach}\left(\frac{|x|+|y|}{|x+y|}\right).$$

End of Lecture 12

Round-Off Error Analysis

We showed that the relative error is roughly bounded by:

$$2\varepsilon_{mach}\left(\frac{|x|+|y|}{|x+y|}\right)$$

Case (1): x, y same signs $\rightarrow$ relative error $\approx 2\epsilon_{\text{mach}}$

Relative error will be rather small.

Case (2): x, y opposite signs. It is easily possible that $|x+y| << |x|+|y|$

Relative error can become very large!

Even subtracting 2 numbers can produce surprisingly **large relative errors**.

Stability of Algorithms

Another way that numerical algorithms can go wrong is called *numerical instability*.

Some algorithms cause small initial errors (e.g., from floating point) to be magnified over time, and thus destroy your solution.

This is true even if the algorithm would be correct under real arithmetic!

Let's see an example.

Stability of Algorithms

Consider solving this integral:

$$I_n = \int_0^1 \frac{x^n}{x+\alpha}\, dx, \quad \text{for some constant } \alpha > 0.$$

For $n=0$, $I_0 = \int_0^1 \frac{1}{x+\alpha}\, dx = \log\left(\frac{1+\alpha}{\alpha}\right)$

For $n>0$, observe,

$$I_n + \alpha I_{n-1} = \int_0^1 \left(\frac{x^n}{x+\alpha} + \alpha \frac{x^{n-1}}{x+\alpha}\right) dx$$

$$= \int_0^1 \frac{x^{n-1}(x+\alpha)}{x+\alpha}\, dx = \int_0^1 x^{n-1}\, dx = \frac{1}{n}$$

Stability of Algorithms

∴ recursive rule:

$$I_n = \frac{1}{n} - \alpha I_{n-1}, \quad \text{with} \quad I_0 = \log\left(\frac{1+\alpha}{\alpha}\right).$$

Evaluate for $n=100$ with

$\alpha = \frac{1}{2} \longrightarrow 6.64 \times 10^{-3}$

$\alpha = 2 \longrightarrow 2.1 \times 10^{22}$

under floating point.

Are these reasonable?

If $\alpha > 1$, then $x + \alpha > 1$ (since $x \in [0,1]$).

So,

$$\int_0^1 \frac{x^n}{x+\alpha}\,dx < \int_0^1 \frac{x^n}{1}\,dx = \frac{1}{n+1}$$

$\therefore\ I_{100} < \frac{1}{101}$ for $\alpha > 1$

Our result for $\alpha = 2$ is wrong, but $\alpha = \frac{1}{2}$ result is correct!

Same algorithm is right for $\alpha = 1/2$, but wrong for $\alpha = 2$. Why?

Suppose fl(I_0) has error E_0 at the start. Assume no other error occurs.

Let $(I_n)_A$ = approx/numerical value of I_n.

Let $(I_n)_E$ = exact value of I_n.

Then $E_n = (I_n)_A - (I_n)_E$ = (signed) error of computing I_n due to E_0.

Exact I_n satisfies $(I_n)_E = \frac{1}{n} - \alpha (I_{n-1})_E$ (1)

Approx I_n satisfies $(I_n)_A = \frac{1}{n} - \alpha (I_{n-1})_A$ (2)

$$(2) - (1) = (I_n)_A - (I_n)_E$$

$$= \left(\frac{1}{n} - \frac{1}{n}\right) - \alpha\left((I_{n-1})_A - (I_{n-1})_E\right)$$

$$E_{n-1}$$

$$E_n = -\alpha\, E_{n-1}$$

$$= (-\alpha)(-\alpha)\, E_{n-2}$$

$$= (-\alpha)^3\, E_{n-3}$$

$$\vdots$$

$$= (-\alpha)^n\, E_0$$

$$\Longrightarrow E_n = (-\alpha)^n\, E_0$$

$$\Longrightarrow |E_n| = |\alpha|^n \cdot |E_0|$$

Algorithm is stable if $|\alpha| < 1$, since E_0 is scaled down in magnitude.

Unstable if $|\alpha| > 1$, since error grows and eventually overwhelms the true result with "garbage".